

Atividade Remota — Máquinas de Turing

Teoria da Computação — Profª Kadidja Valéria

Data: 29/09/2026

Etapa 1 — Introdução

1. O que é uma Máquina de Turing?

É um modelo matemático que o Alan Turing criou em 1936 para entender, de forma bem precisa, o que significa "calcular" alguma coisa. Ela não é uma máquina de verdade que alguém vai montar, e sim uma ideia no papel: uma fita onde ficam os símbolos, uma cabeça que lê e escreve nessa fita e um conjunto de regras que dizem o que fazer a cada passo. Mesmo sendo simples, ela consegue fazer qualquer cálculo que um computador moderno faz.

2. Quais são os principais componentes de uma Máquina de Turing?

- **Fita:** é como se fosse a memória. Ela é dividida em células, cada uma com um símbolo, e é infinita.
- **Cabeça de leitura e escrita:** fica em cima de uma célula por vez. Ela lê o símbolo, pode trocar por outro e anda uma casa para a esquerda ou para a direita.
- **Estados:** mostram em que "momento" do trabalho a máquina está, como se fosse o que ela está lembrando naquele instante.
- **Alfabeto:** são os símbolos que podem aparecer na fita (no meu caso: 0, 1, X, Y e o branco).
- **Regras de transição:** dizem o que a máquina faz de acordo com o estado atual e o símbolo que ela leu.
- **Estado inicial e estados finais:** mostram onde o processamento começa e quando ele termina, aceitando ou rejeitando a entrada.

3. Qual é a importância das Máquinas de Turing para a computação?

Ela é importante porque foi a partir dela que deu para definir, de um jeito formal, o que um computador consegue ou não consegue calcular. Ela é uma das bases da Ciência da Computação e ajudou a formar as ideias de algoritmo e de computabilidade. Pelo vídeo também deu para entender que é isso que separa uma calculadora de um computador: a calculadora só faz as operações que já vêm nela, enquanto um computador, assim como a Máquina de Turing, segue qualquer programa que você der para ele. Essa ideia inspirou a arquitetura de Von Neumann, que é usada até hoje.

4. Qual é a relação entre Máquina de Turing e algoritmo?

Um algoritmo é uma sequência de passos para resolver um problema. A Máquina de Turing consegue executar esses passos através das regras de transição, então ela funciona como um modelo para representar e estudar algoritmos. Na prática, falar que existe um algoritmo para um problema é o mesmo que falar que dá para construir uma Máquina de Turing que resolve esse problema (essa é a ideia da Tese de Church-Turing).

Etapa 2 — Simulação

Descrição da Máquina de Turing criada

Montei a máquina no simulador **turingmachinesimulator.com** para reconhecer palavras do tipo 0^n1^n , ou seja, com a mesma quantidade de 0 e de 1, com todos os 0 vindo antes dos 1.

A ideia é ir "riscando" os símbolos em pares. A máquina marca o primeiro 0 que encontra com um X, anda para a direita até achar o primeiro 1 e marca ele com um Y. Depois ela volta para a esquerda até o último X e repete tudo. Quando não sobra mais nenhum 0, ela passa por cima dos Y para ver se ainda tem algum 1 perdido. Se chegar no final da palavra só com Y, ela aceita. Se em algum momento aparecer um símbolo que não era para estar ali (por exemplo, um 1 sobrando), ela não tem regra para aquilo, então para e rejeita a palavra.

Estados usados: q0 (início de cada rodada), q1 (procurando um 1), q2 (voltando para a esquerda), q3 (conferindo o final) e qA (aceitação).

Tabela de transições

Estado	Lê	Escreve	Move	Vai para	O que significa
q0	0	X	→	q1	Marca um 0 e vai procurar um 1
q0	Y	Y	→	q3	Acabaram os 0: hora de conferir o resto
q1	0	0	→	q1	Passa pelos 0 que ainda faltam
q1	Y	Y	→	q1	Passa pelos 1 já marcados
q1	1	Y	←	q2	Achou um 1, marca e volta
q2	0	0	←	q2	Volta pela fita
q2	Y	Y	←	q2	Volta pela fita
q2	X	X	→	q0	Chegou no último X, começa outra rodada
q3	Y	Y	→	q3	Confere se só sobrou Y
q3	—	—	—	qA	Chegou no fim sem sobras: ACEITA

Código usado no simulador

```
name: 0n1n
init: q0
accept: qA
```

```
q0,0
q1,X,>
q0,Y
q3,Y,>
```

```
q1,0
q1,0,>
q1,Y
q1,Y,>
q1,1
q2,Y,<
```

```
q2,0
q2,0,<
q2,Y
q2,Y,<
```

```

q2,X
q0,X,>

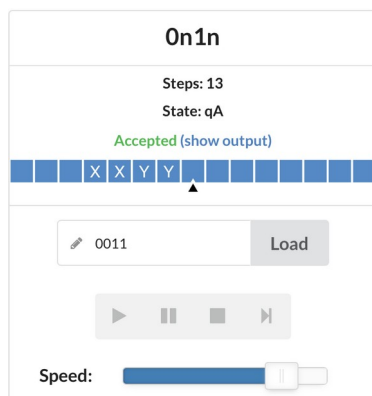
q3,Y
q3,Y,>
q3,_
qA,_,-

```

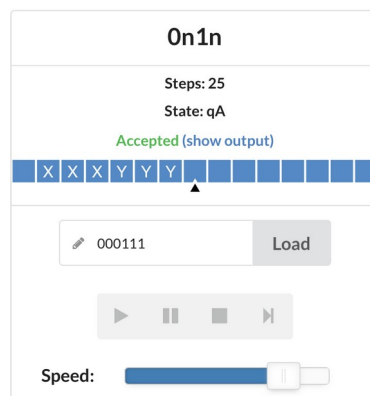
Etapa 3 — Registro da simulação

Teste	Entrada	Esperado	Obtido	Passos	Estados percorridos
1	0011	ACEITA	ACEITA	13	$q_0 \rightarrow q_1 \rightarrow q_1 \rightarrow q_2 \rightarrow q_2 \rightarrow q_0 \rightarrow q_1 \rightarrow q_1 \rightarrow q_2 \rightarrow q_2 \rightarrow q_0 \rightarrow q_3 \rightarrow q_3 \rightarrow q_A$
2	000111	ACEITA	ACEITA	25	$(q_0 \rightarrow q_1 \rightarrow q_1 \rightarrow q_1 \rightarrow q_2 \rightarrow q_2 \rightarrow q_2)$ repetido 3 vezes $\rightarrow q_0 \rightarrow q_3 \rightarrow q_3 \rightarrow q_3 \rightarrow q_A$
3	00111	REJEITA	REJEITA	12	$q_0 \rightarrow q_1 \rightarrow q_1 \rightarrow q_2 \rightarrow q_2 \rightarrow q_0 \rightarrow q_1 \rightarrow q_1 \rightarrow q_2 \rightarrow q_2 \rightarrow q_0 \rightarrow q_3 \rightarrow q_3$ (parou lendo 1)

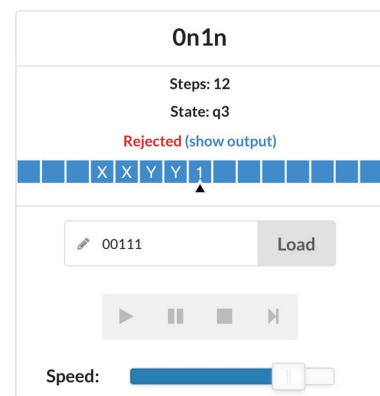
Capturas de tela



Teste 1 — 0011 (Accepted)



Teste 2 — 000111 (Accepted)



Teste 3 — 00111 (Rejected)

Interpretação dos resultados

Os três testes deram o resultado que eu esperava. No teste 1 (0011) e no teste 2 (000111), a máquina conseguiu formar um par para cada 0 e cada 1, então no final a fita ficou só com X e Y e ela terminou no estado qA, aceitando a palavra. Dá para ver também que quanto maior a palavra, mais passos ela leva (13 para 0011 e 25 para 000111), porque a cabeça precisa ir e voltar pela fita a cada par.

Já no teste 3 (00111) tem um 1 a mais do que 0. Depois de formar os dois pares, sobrou um 1 sem par. A máquina estava no estado q3, que só espera encontrar Y ou o fim da palavra, e como leu um 1 ela parou sem chegar em qA. Por isso a palavra foi rejeitada, que era exatamente o esperado.

Etapa 4 — Reflexão sobre os limites computacionais

Não, uma Máquina de Turing não consegue resolver qualquer problema. Parece estranho, porque ela consegue fazer tudo o que um computador faz, mas justamente por isso ela também tem os mesmos limites. O exemplo mais conhecido é o Problema da Parada: não existe nenhum algoritmo que consiga olhar para qualquer programa e dizer com certeza se ele vai terminar ou se

vai rodar para sempre. O Turing provou isso mostrando que, se esse "verificador" existisse, daria para criar um programa que usa o verificador contra ele mesmo, e isso gera uma contradição. Ou seja, não é falta de tecnologia ou de computador mais rápido, é uma impossibilidade lógica. Como qualquer computador é, no fundo, equivalente a uma Máquina de Turing, esse limite vale para todos eles.

Questão final

Primeiro é importante separar duas coisas: um problema difícil e um problema impossível. Um problema difícil tem solução, existe um algoritmo para ele, mas esse algoritmo pode demorar muito, às vezes anos, para terminar. Já um problema impossível (indecidível) não tem nenhum algoritmo que funcione para todos os casos, não importa quanto tempo ou quanto poder de processamento a gente tenha.

O fato de eu não conseguir resolver um problema não prova que ele é impossível, pode ser só que ainda não encontrei o caminho. Para ter certeza, é preciso uma prova. A forma mais comum é a redução: eu mostro que, se o meu problema tivesse solução, eu conseguiria usar essa solução para resolver o Problema da Parada. Como já foi provado que o Problema da Parada não tem solução, o meu problema também não pode ter. Pela Tese de Church-Turing, se nenhuma Máquina de Turing resolve, então nenhum computador resolve.

Por outro lado, se eu consigo montar uma Máquina de Turing que sempre para e dá a resposta certa, como a que fiz para 0^n1^n , então o problema é computável, mesmo que seja trabalhoso. Nesse caso, a pergunta passa a ser sobre eficiência, e entra a área de complexidade (problemas como os NP-completos, que têm solução, mas até hoje não se conhece um jeito rápido de resolver).